Lecture 9

Exception

Exception

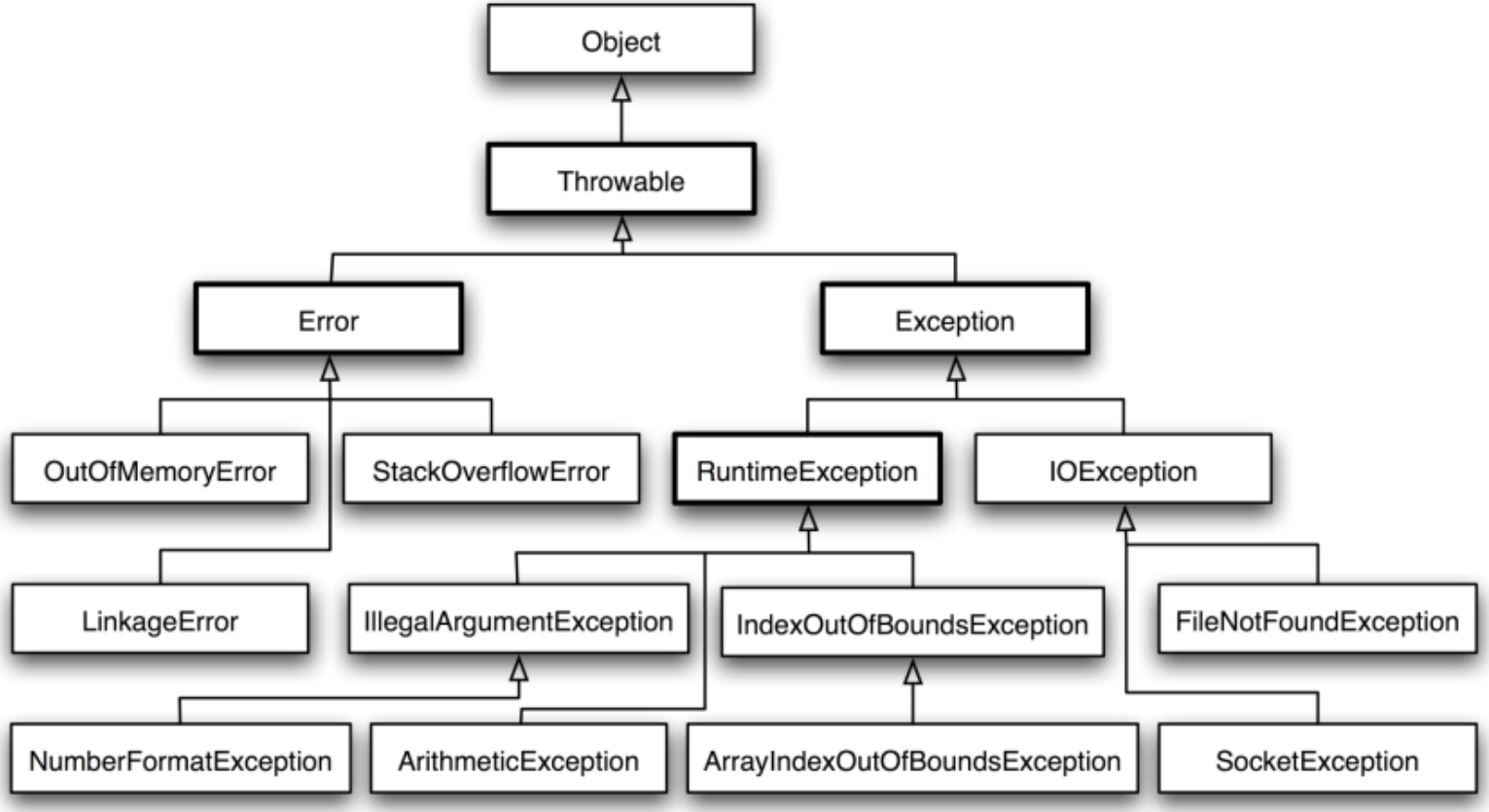
- Exceptions are errors that:
  - appear when the program is running
  - cause the program to halt and display an error message
- Example:
  - When dividing 2 integers, if we set the denominator to 0, an error (typed `ArithmeticException`) occurs.

int a = 5, b = 0;

a/b ⇨ error !

Exception

- The `Throwable` class handles errors and exceptions (`Error`, `Exception`).



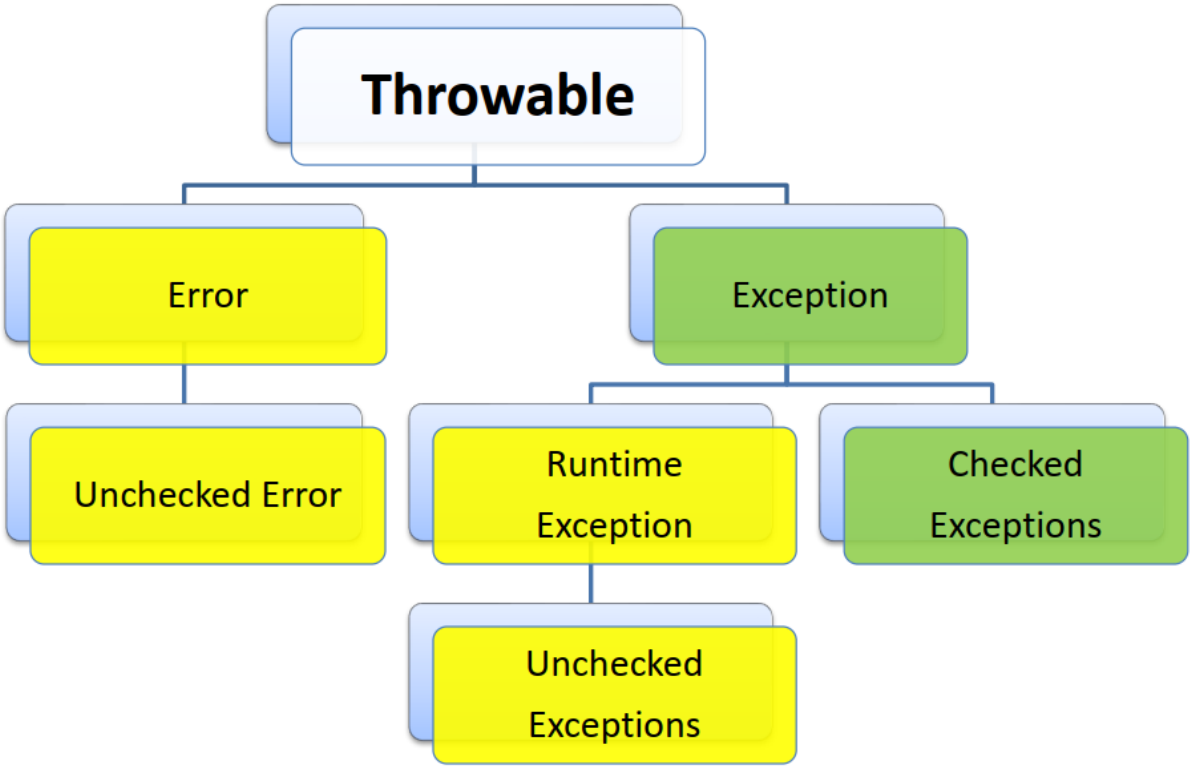
Exception Example

- Convert String to number

```
try {
    String string = "12";
    int number = Integer.parseInt(string);
} catch (Exception e) {
    System.out.println("Error");
}
```

Exception classification

- Exceptions are divided into two types: checked (green) and unchecked (yellow).



Exception classification

- Unchecked exceptions:
  - These are exceptions checked at runtime.
  - They include the `Error` class, the `RuntimeException` class, and their subclasses.
  - For example: `Integer.parseInt("abc")` compiles but throws an error when executed.
- Checked exceptions:
  - These are exceptions checked at compile time.
  - They include all other exception classes.
  - For example: `new FileWriter("c:/data.txt")` results in a compilation error even if the file exists.

Exception classification

- Checked exceptions:
  - `ClassNotFoundException`
  - `IOException`
    - `FileNotFoundException`
    - `EOFException`
- Unchecked exceptions:
  - `ArithmeticException`
  - `IllegalArgumentException`
  - `IndexOutOfBoundException`
  - `NullPointerException`
  - `InputMismatchException`

Exception handling

- To handle exceptions using `try…catch` blocks

```
try {
    // statements
} catch (IllegalArgumentException ex) {
    // the exception handling code
}
```

Example

```java
try {
    // this will throw an ArithmeticException
    int result = 10 / 0;
} catch (ArithmeticException e) {
    // code to handle the exception
    System.out.printf("An arithmetic error occurred: "
            + e.getMessage());
}
```

Consider the situation

- Assuming there's a `String` array `ss`. What errors might occur with the following command?

```
Integer.parseInt(ss[5]);
```

- Clearly, there are possibilities of three errors:
  - The `ss` array is `null` (not initialized).
  - The `ss` array has fewer than 6 elements.
  - The 6th element (`ss[5]`) cannot be converted to a number.

➢ How would you handle these errors?

Catch detailed errors

- The `try` code block can have many exceptions. Use multiple `catch` blocks to catch and handle the details of those exceptions.

```
String[] ss = {"1", "a", "2", "c", "d"};
try {
    int a = Integer.parseInt(ss[5]);
} catch (NumberFormatException e1) {
    System.out.println("Incorrect number format");
} catch (ArrayIndexOutOfBoundsException e2) {
    System.out.println("Outside of array bound");
} catch (NullPointerException e3) {
    System.out.println("Uninitialized Array");
}
```

Catching common errors

- The second `catch` block in the following code snippet catches both the `NumberFormatException` and `NullPointerException` exceptions because both of these exceptions are subclasses of `Exception`.
- The catch block catching general exceptions must be placed last.

Catching common errors

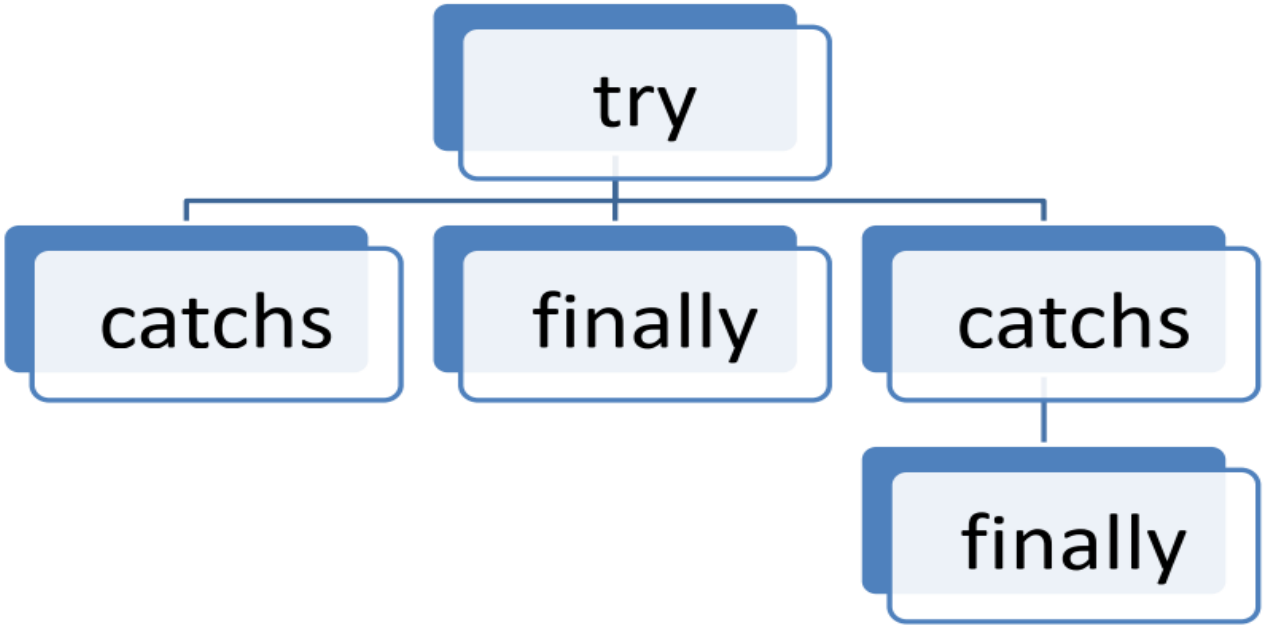
```
String[] ss = {"1","a","2","c","d"};
try{
    int a = Integer.parseInt(ss[5]);
}
catch(NumberFormatException e1){
    System.out.println("Incorrect number format");
}
catch(Exception e2){
    System.out.println("Error");
}
```

Example: Converting date

```
String dateString = "2024-03-16";
DateFormat dateFormat = new SimpleDateFormat( pattern: "yyyy-MM-dd");
try {
    Date date = dateFormat.parse(dateString);
    System.out.println("Converted date: " + date);
} catch (Exception e) {
    System.out.println("Error parsing date: " + e.getMessage());
}
```

Using `finally`

- Every `try` block requires at least one `catch` block and/or exactly one `finally` block.
- The `finally` block will be executed regardless of whether an exception is caught or not.



Using `finally`

```
try{
    int a = Integer.parseInt(ss[1]);
}
catch (Exception e){
    System.out.println("Error");
}
```

```
try{
    int a = Integer.parseInt(ss[1]);
}
catch (Exception e){
    System.out.println("Error");
}
finally {
    ss = null;
}
```

The exception has been handled

Using `finally`

When the exception has not been handled, the code inside finally still gets executed before the program terminates.

```
try{
    int a = Integer.parseInt(ss[1]);
}
finally {
    ss = null;
}
```

Using `throws` and `throw`

- `throw` is used to generate an exception.

```
throw new RuntimeException("Error");
```

- `throws` is used to propagate an exception out of the method. The exception will be handled when the method is called.

```
void method() throws FileNotFoundException {
...
}
```

Using `throws`

```java
public static void writeFile() throws IOException {
    FileWriter file = new FileWriter( fileName: "data.txt");
    file.write( str: "Handle Exception in Java");
    file.write( c: 100);
    System.out.println("Wrote");
    file.close();
}
```

```java
try{
    Demo obj = new Demo();
    obj.writeFile();
} catch (IOException e) {
    System.out.println("Error"+ e);
}
```

Using `throws`

- If when calling the method there is an exception that you do not want to handle, you can continue to throw it out.

```
public static void writeFile() throws IOException {
    FileWriter file = new FileWriter( fileName: "data.txt");
    file.write( str: "Handle Exception in Java");
    file.write( c: 100);
    System.out.println("Wrote");
    file.close();
}

public static void main(String[] args) throws IOException {
    Demo obj = new Demo();
    obj.writeFile();
    System.out.println("Using the throws keyword");
}
```

Using `throw`

- Normally, exceptions will be “thrown” by the Java runtime system. However, we can still program to `throw` exceptions when encountering certain situations during programming.
- Within a method, it's possible to throw multiple exceptions.
- There are two ways to "throw" exceptions:
  - Using the `new` operator
  - Passing a parameter to the `catch` clause.
- For example:

```
if (check == 0)
    throw new NullPointerException();
```

Using `throw`

```
1 usage
static void demoProc(){
    try{
        throw  new NullPointerException("demo");
    }catch(NullPointerException e){
        System.out.println("Handle exception in demoPro");
        throw e;
    }
}

public static void main(String[] args) {
    try {
        demoProc();
    }catch (NullPointerException e){
        System.out.println("Handle exception in main ");
    }
}
```

Creating new exceptions

- We can create our own exception handling class by sub-classing either a checked or unchecked Exception class.

```
public class MyException extends Exception{
    2 usages
    private  int message;
    MyException(int a){
        message = a;
    }

    @Override
    public String toString() {
        return "MyException{" +
                "message=" + message +
                '}';
    }
}
```

Creating new exceptions

- After creating a custom Exception, we can use it just like any pre-defined Exception.

```
2 usages
static  void calculator(int a) throws MyException{
    if(a > 10){
        throw new MyException(a);
    }
    System.out.println("Normal exit");
}

public static void main(String[] args) {
    try{
        calculator( a: 1); // Don't create exception
        calculator( a: 20); // create exception
    } catch (MyException e){
        System.out.println("Caugh" + e);
    }
}
```

Summary

- Exception
- Hierarchy of exceptions
- Exception handling
- Using `finally`
- Using `throws` and `throw`